

Bienvenue au cours Angular Avancé

- Nom de l'enseignant : Ghislain Vachet
- Promo : **M PRO LEAD DEV XPERT IPI**
- Classe : M1 XCYBR/M1 XDEV G2
- Dates : 26 Novembre 2025



Objectifs du cours et contexte pédagogique

Objectifs généraux d'apprentissage :

- Concevoir et développer des **applications Angular complexes et maintenables**
- Implémenter des **architectures robustes** avec **RxJS** et la **gestion avancée des états**
- Utiliser des techniques de **Lazy Loading**, **Dynamic Components** et **Angular Universal**
- Maîtriser les **tests unitaires et d'intégration** avec **Jasmine** et **Karma**
- Optimiser la **performance** et la **sécurité** des applications Angular
- Déployer efficacement des applications Angular en **environnement de production**



Blocs RNCP associés :

C2.2.3 (BC02C05) —
principal ; C2.2.1 (BC02C03)



Informations - Evaluations

Informations – Évaluations

Modalités pédagogiques : Cours pratique, démonstrations et TPs sur projet fil rouge (*TaskBoard Pro*).

Évaluation :

- 50 % contrôle continu (quiz, TPs)
- 50 % projet final (livrable GitHub)

Ressources : Angular CLI, Node.js, VS Code, GitHub.



Objectifs de la séance

- Révision du cycle de vie Angular 17+
- Création d'un projet structuré avec Angular CLI
- Utilisation des composants standalone et routing de base



Qu'est-ce qu'un SPA

SPA (Single-Page Application) = application web qui **charge une fois** la page, puis **met à jour la vue côté client** (JavaScript) **sans rechargement complet**.

Le routeur (ex. Angular Router) **change l'URL** et **remplace le composant affiché** dans un `<router-outlet>`.

Avantages :

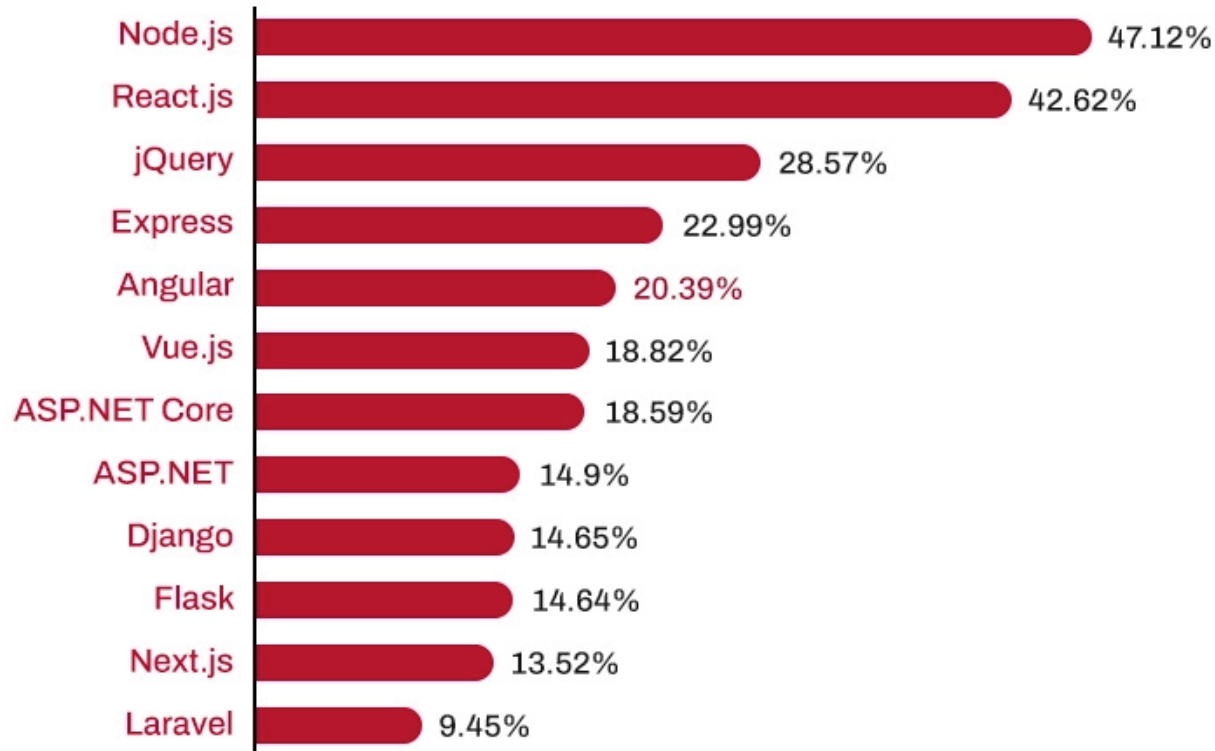
- Navigation **fluide** (moins de latence, pas de flash écran).
- **Réutilisation** des composants / état client.
- **Moins** d'allers-retours serveur pour le rendu.

Points d'attention :

- **SEO** et **temps de 1er rendu** → mitigés par **SSR/Pre-render** (Angular Universal).
- **Bundle** côté client → nécessite **optimisation** (lazy, code-splitting).



Angular statistique



Pourquoi Angular ?

- **Cadre complet et cohérent** : routing, formulaires, tests intégrés
- **TypeScript natif** : typage, refactoring sûr
- **Architecture claire** : standalone components, structure prévisible
- **Soutenu par Google** : équipe dédiée, roadmap stable, MAJ régulières
- **Prêt pour l'entreprise** : maintenabilité, normes, outillage CLI



Javascript vs TypeScript 1/2

- JavaScript : typage dynamique, erreurs à l'exécution
 - TypeScript : typage statique, vérification avant exécution
- Angular utilise TypeScript par défaut pour plus de fiabilité.



Javascript vs TypeScript 2/2

- TypeScript **n'est pas compris directement par le navigateur :**
- il doit d'abord être **transpilé (converti)** en JavaScript.
- Cette étape de compilation permet de **vérifier le code et de signaler les erreurs** avant son exécution réelle.



Exemple : Typage en action

```
function add(a: number, b: number): number {  
  return a + b;  
}
```

add(3, 4); // OK

add(3, '4'); // Erreur détectée avant exécution



Comprendre la structure Angular

```
taskboard-pro/  
└─ src/  
    └─ app/  
        └─ features/           # Composants liés à une fonctionnalité  
        └─ shared/            # Éléments réutilisables (UI, pipes...)  
        └─ app.ts              # Composant racine  
        └─ app.routes.ts      # Configuration du routage  
  
    └─ assets/                # Images, icônes, fichiers statiques  
    └─ environments/          # Variables d'environnement  
    └─ main.ts                # Point d'entrée de l'application  
  
└─ angular.json              # Configuration du projet Angular  
└─ package.json              # Dépendances et scripts NPM  
└─ tsconfig.json             # Configuration TypeScript
```



Explication Structure Angular

Élément	Rôle	Exemple / métaphore
<code>main.ts</code>	Point d'entrée, démarre l'app avec <code>bootstrapApplication()</code>	Le contacteur qui allume la maison
<code>app.ts</code>	Composant racine, premier affiché dans le DOM	Le hall d'entrée de l'application
<code>features/</code>	Dossiers contenant les vraies pages/fonctionnalités	Les pièces principales
<code>shared/</code>	Composants communs (header, boutons, pipes)	Les meubles partagés
<code>assets/</code>	Images, icônes, CSS global	Le grenier
<code>angular.json</code>	Fichier de configuration des builds	Le plan de la maison



Les 3 Piliers d'Angular moderne

- 1 TypeScript → code lisible et sûr
- 2 Standalone Components → simplicité et modularité
- 3 Structure standardisée → cohérence entre projets



Créer un projet Angular 17+

- `ng new taskboard-pro --routing --style=css`
- `cd taskboard-pro`
- `ng serve`



Créer un composant 1/2

ng generate component header

ou plus court :

ng g c header



Créer un composant 2/2

Étape	Action effectuée par Angular CLI
	Crée un dossier <code>src/app/header/</code>
	Génère 4 fichiers : <code>header.ts</code> , <code>.html</code> , <code>.scss</code> , <code>.spec.ts</code>
	Met à jour le projet pour inclure le nouveau composant
	Prépare le composant à être importé et affiché dans l'app

Comment l'utiliser ?

1.Importer le composant dans app.ts

```
import { Header } from './header/header';
```

```
@Component({  
  selector: 'app-root',  
  standalone: true,  
  imports: [Header], // ← Obligatoire !  
  templateUrl: './app.html',  
  styleUrls: ['./app.css']  
})
```

2.Ajouter dans app.html <app-header></app-header> Angular reconnaît la **balise app-header** grâce au selector défini dans @Component.

Chaque composant Angular est une brique indépendante, avec son propre code, son template et son style.



Cycle de vie d'un composant Angular

Étape	Méthode	Description rapide
1	ngOnChanges()	Réagit aux changements d'un ou plusieurs @Input().
2	ngOnInit()	Initialisation du composant (idéal pour charger des données).
3	ngDoCheck()	Détection manuelle de changements (rarement utilisée).
4	ngAfterViewInit()	Exécutée après le rendu du template (DOM enfant est disponible).
5	ngAfterViewChecked()	Appelé après chaque MAJ du DOM (utilise mais rarement utilisé).
6	ngOnDestroy()	Nettoyage (désabonnements, timers, listeners, etc.).



Exemple d'utilisation

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-home',  
  template: `  
    <h2>Accueil</h2>  
    <p>Bienvenue sur la page d'accueil.</p>  
  `,  
})
```

```
export class Home { //On n'importe plus OnInit ni OnDestroy depuis @angular/core.
```

```
  ngOnInit() { //démontre lorsque le composant est affiché dans le DOM,  
    console.log('Composant initialisé');  
  }
```

```
  ngOnDestroy() { //se termine dès que le composant est retiré du DOM —par exemple, lorsqu'on change de route.  
    console.log('Composant détruit');  
  }  
}
```



Mettre en place le routing de base

Angular est une SPA : le backend renvoie toujours index.html.
Le routing permet à Angular d'afficher la bonne page (Home, About...) selon l'URL, sans recharger l'application.

Comment le routing fonctionne dans Angular ?

- Le routing relie une URL à un composant.
- Il permet de naviguer sans recharger la page.
- Angular gère le routage côté client.

💡 Une seule application → plusieurs vues affichées dynamiquement.



Les éléments du routing

Étape	Élément	Rôle / explication
1 <code>app.routes.ts</code>	Définition des routes	"On y définit la liste des pages de l'application (ex. Home, About)."
2 <code>provideRouter()</code>	Active le routeur dans <code>main.ts</code> (via <code>app.config.ts</code>)	"Il relie Angular au système de routage pour qu'il sache gérer les URL."
3 <code>RouterLink</code>	Directive de navigation (équivalent du lien <code><a></code>)	"Permet à l'utilisateur de changer de route sans recharger la page."
4 <code>RouterLinkActive</code>	Directive de style dynamique	"Ajoute une classe CSS (souvent <code>active</code>) au lien correspondant à la route affichée."
5 <code>RouterOutlet</code>	Directive d'affichage dynamique	"C'est la zone où Angular insère automatiquement le composant correspondant à la route active."



Exemple1/3

```
ng g c home  
ng g c about
```

src/app/app.routes.ts

```
import { Routes } from '@angular/router';  
import { Home } from './home/home';  
import { About } from './about/about';
```

```
export const routes: Routes = [  
  { path: '', component: Home },  
  { path: 'about', component: About },  
];
```



Exemple2/3

src/app/app.ts

```
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';
```

```
@Component({
```

```
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
```

```
  template: `
```

```
    <nav style="padding:.5rem; border-bottom:1px solid #ddd;">
```

```
      <a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{ exact: true }">Accueil</a> |
```

```
      <a routerLink="/about" routerLinkActive="active">À propos</a>
```

```
    </nav>
```

```
    <main style="padding:1rem;">
```

```
      <router-outlet></router-outlet> // est utilisé une seule fois dans App :c'est la zone où Angular affiche le composant correspondant à la route active."
```

```
    </main>
```



Exemple3/3

src/app/home/home.ts

template: `

<h2>Accueil</h2>

<p>Bienvenue sur la page d'accueil.</p>

`
;

src/app/about/about.ts

template: `

<h2>À propos</h2>

<p>Infos sur l'application.</p>

`
;



Fichiers principaux à configurer

- **app.routes.ts**

Définit les routes de l'application (" pour Home, 'about' pour About).

- **app.ts**

Contient la structure principale : navigation et <router-outlet>.

- **home.ts**

Définit le contenu de la page d'accueil.

- **about.ts**

Définit le contenu de la page "À propos".

- **index.html**

Contient <base href="/"> indispensable pour le fonctionnement du router Angular.



Ancrage projet fil rouge

On applique la théorie à notre projet :
TaskBoard Pro



TP démarrage du fil rouge

Objectif du TP :

Créer (seul ou en binôme) le squelette de l'application **TaskBoard Pro** avec un routing opérationnel

Attendus techniques

- Projet Angular 20 : `ng new taskboard-pro --routing --style=scss ou css`
- Pages : Home (/) et About (/about)
- **Routing en place :**
 - `app.routes.ts` : routes vers Home et About
 - `app.ts` : `<router-outlet>` + menu de navigation (`RouterLink`, `RouterLinkActive`)
 - `app.config.ts` : `provideRouter(routes)`
 - `index.html` : `<base href="/">`

Structure dossiers (minimum) : `src/app/`

home/
about/

Livrables

- Dépôt GitHub nommé `taskboard-pro`
- README (2 lignes : commandes utilisés & routes fonctionnelles)

Lien du form : <https://docs.google.com/forms/d/e/1FAIpQLSei4nEB4NUdo-lhgf7829knSaAFLXGZ7CNV78Yuqzj-CbEECA/viewform?usp=header>

